



Guía de Autenticación - Trading Academy

Fecha de Investigación: 2 de mayo de 2026
Aplicación: Next.js Trading Academy
Estado: Sistema completamente documentado y usuarios de prueba creados



Tabla de Contenidos

- 1. [Usuarios Existentes en la Base de Datos](#)
- 2. [Flujo de Autenticación](#)
- 3. [Sistema de Roles](#)
- 4. [Credenciales de Acceso](#)
- 5. [Instrucciones de Login - Administrador](#)
- 6. [Instrucciones de Login - Cliente Regular](#)
- 7. [Arquitectura Técnica](#)



Usuarios Existentes en la Base de Datos

Estado Actual de la Base de Datos

La aplicación utiliza **PostgreSQL** con **Prisma ORM**. Se han creado dos usuarios de prueba:

Email	Rol	Nombre Completo	Estado	Fecha de Creación
ad-min@tradingacademy.com	admin	Admin Trading Academy	Verificado	2026-05-02 01:18:06
usuario@tradingacademy.com	user	Juan Cliente	Verificado	2026-05-02 01:18:07

Modelo de Usuario en la Base de Datos

```
User {
  id: String (único)
  email: String (único)
  password: String (hash bcryptjs)
  firstName: String
  lastName: String
  whatsappNumber: String
  role: String (default: "user")
  emailVerified: DateTime (requerido para login)
  verificationToken: String
  tokenExpiry: DateTime
  createdAt: DateTime
  updatedAt: DateTime
}
```

Flujo de Autenticación

Diagrama del Flujo

- FLUJO DE AUTENTICACIÓN
1. REGISTRO (Opcional - si no existe usuario)
 - ↳ POST /api/signup
 - ├ Validar email y contraseña
 - ├ Generar contraseña temporal
 - ├ Crear usuario en BD (emailVerified = null)
 - ├ Enviar email de verificación
 - └ Usuario recibe contraseña por WhatsApp
 2. VERIFICACIÓN DE EMAIL
 - ↳ GET /verify-email?token={verificationToken}
 - ├ Validar token
 - ├ Marcar emailVerified = true
 - └ Redirigir a login
 3. LOGIN
 - ↳ NextAuth.js Credentials Provider
 - ├ POST /api/auth/signin (NextAuth endpoint)
 - ├ CredentialsProvider valida:
 - ├ Email existe
 - ├ Email está verificado (emailVerified ≠ null)
 - └ Contraseña coincide (bcrypt.compare)
 - ├ Generar JWT token
 - ├ Guardar sesión en cliente (httpOnly cookie)
 - └ Redirigir según rol:
 - ├ Si role = "admin" → Dashboard Admin
 - └ Si role = "user" → Dashboard Cliente
 4. PROTECCIÓN DE RUTAS
 - ↳ Middleware requireAdminSession()
 - ├ Obtener sesión (getSession)
 - ├ Validar role === "admin"
 - └ Si no: redirigir a /dashboard

Pasos Detallados de Login

Paso 1: Visitar la página de login

URL: <https://d4896f84e.preview.abacusai.app/login>

Paso 2: Completar formulario

- **Email:** Tu dirección de correo (del usuario)
- **Contraseña:** Tu contraseña asignada

Paso 3: Envío de credenciales

- NextAuth.js utiliza **Credentials Provider**
- Las credenciales se validan contra la base de datos PostgreSQL
- Se verifica:
 - El usuario existe (email único)
 - El email está verificado (`emailVerified` $\neq$ null)
 - La contraseña es correcta (bcrypt hashing)

Paso 4: Generación de sesión

- Se genera un **JWT token** con:
 - ID del usuario
 - Email
 - Nombre completo
 - Rol (admin o user)

Paso 5: Redirección según rol

- **Admin:** Redirige a `/dashboard/admin`
- **Usuario Regular:** Redirige a `/dashboard`

Sistema de Roles

Roles Disponibles

1. ADMIN (`role = "admin"`)

Acceso:

- ☒ Panel de administración completo
- ☒ Gestión de usuarios
- ☒ Gestión de cursos
- ☒ Gestión de leads y contactos
- ☒ Configuración de bots IA
- ☒ Analytics y reportes
- ☒ Configuraciones de seguridad

URL de Acceso:

<https://d4896f84e.preview.abacusai.app/dashboard/admin>

Submódulos:

- /dashboard/admin/users - Gestión de usuarios y roles
- /dashboard/admin/leads - Seguimiento de leads
- /dashboard/admin/courses - Catálogo de cursos
- /dashboard/admin/bots - Gestión de bots IA
- /dashboard/admin/content - Contenido del sitio
- /dashboard/admin/notifications - Notificaciones
- /dashboard/admin/payments - Información de pagos
- /dashboard/admin/security - Configuraciones de seguridad

Protección:

```
// En: /lib/admin.ts
export async function requireAdminSession() {
  const session = await getServerSession(authOptions);

  if (!session) {
    redirect("/login");
  }

  if ((session.user as any)?.role !== "admin") {
    redirect("/dashboard");
  }

  return session;
}
```

2. USER (role = "user" - Default)**Acceso:**

- ☒ Dashboard de cliente
- ☒ Visualización de cursos asignados
- ☒ Seguimiento de progreso
- ☒ Chat con tutor IA
- ☒ Perfil personal

URL de Acceso:

<https://d4896f84e.preview.abacusai.app/dashboard>

Submódulos:

- /dashboard/mis-cursos - Mis cursos activos
 - /dashboard/curso/[courseId] - Contenido del curso
 - Tutor IA chat integrado
-

Credenciales de Acceso

Usuario Administrador

Campo	Valor
Email	admin@tradingacademy.com
Contraseña	Admin@123456
Nombre	Admin Trading Academy
Rol	admin
Teléfono WhatsApp	+5215555555555
Estado Email	☒ Verificado

Usuario Cliente Regular

Campo	Valor
Email	usuario@tradingacademy.com
Contraseña	Usuario@123456
Nombre	Juan Cliente
Rol	user
Teléfono WhatsApp	+5214444444444
Estado Email	☒ Verificado

LOGIN ADMINISTRADOR

Paso a Paso en Español

Paso 1: Abrir la aplicación

1. Ve a: <https://d4896f84e.preview.abacusai.app>
2. Haz clic en **“Iniciar Sesión”** en la barra de navegación
 - O ve directamente a: <https://d4896f84e.preview.abacusai.app/login>

Paso 2: Completar el formulario de login

1. En el campo **Email**, escribe:

admin@tradingacademy.com
2. En el campo **Contraseña**, escribe:

Admin@123456

3. Haz clic en el botón **“Iniciar Sesión”**

Paso 3: Acceso al panel administrativo

- La aplicación te redirigirá automáticamente al:
`https://d4896f84e.preview.abacusai.app/dashboard/admin`

Paso 4: Usar el panel administrativo

Una vez dentro del panel administrativo, verás el menú con las siguientes opciones:



Gestión de Usuarios y Roles

- URL: `/dashboard/admin/users`
- Aquí puedes:
- Ver todos los usuarios registrados
- Crear nuevos usuarios
- Editar roles (cambiar entre “admin” y “user”)
- Desactivar o eliminar usuarios



Gestión de Leads

- URL: `/dashboard/admin/leads`
- Aquí puedes:
- Ver contactos que se han registrado
- Hacer seguimiento de leads
- Ver historial de contactos



Gestión de Cursos

- URL: `/dashboard/admin/courses`
- Aquí puedes:
- Crear nuevos cursos
- Editar contenido de cursos
- Publicar o despublicar cursos
- Asignar cursos a usuarios
- Establecer precios



Gestión de Bots IA

- URL: `/dashboard/admin/bots`
- Aquí puedes:
- Configurar bots de IA
- Ver planes de bots
- Activar/desactivar bots



Contenido del Sitio

- URL: `/dashboard/admin/content`
- Aquí puedes:
- Editar secciones del sitio web
- Gestionar testimonios
- Actualizar preguntas frecuentes (FAQ)



Notificaciones

- URL: `/dashboard/admin/notifications`
- Aquí puedes:
- Ver notificaciones del sistema

- Configurar notificaciones
- Enviar notificaciones a usuarios

Información de Pagos

- URL: `/dashboard/admin/payments`
- Aquí puedes:
- Ver transacciones
- Historial de pagos
- Informes de ingresos

Configuraciones de Seguridad

- URL: `/dashboard/admin/security`
- Aquí puedes:
- Cambiar contraseña
- Configurar autenticación de dos factores
- Ver registros de actividad

LOGIN CLIENTE REGULAR

Paso a Paso en Español

Paso 1: Abrir la aplicación

1. Ve a: `https://d4896f84e.preview.abacusai.app`
2. Haz clic en **“Iniciar Sesión”** en la barra de navegación
 - O ve directamente a: `https://d4896f84e.preview.abacusai.app/login`

Paso 2: Completar el formulario de login

1. En el campo **Email**, escribe:
`usuario@tradingacademy.com`
2. En el campo **Contraseña**, escribe:
`Usuario@123456`
3. Haz clic en el botón **“Iniciar Sesión”**

Paso 3: Acceso al dashboard de cliente

- La aplicación te redirigirá automáticamente al:
`https://d4896f84e.preview.abacusai.app/dashboard`

Paso 4: Funcionalidades disponibles como cliente

Mis Cursos

- URL: `/dashboard/mis-cursos`
- Aquí puedes:
- Ver todos los cursos que tienes acceso
- Ver tu progreso en cada curso
- Acceder a las lecciones
- Descargar materiales del curso

Contenido del Curso

- URL: `/dashboard/curso/[courseId]`
- Aquí puedes:

- Ver módulos y lecciones
- Marcar lecciones como completadas
- Acceder a recursos (videos, PDFs, etc.)
- Ver tu progreso general

Tutor IA Chat

- Disponible en el dashboard
- Aquí puedes:
 - Hacer preguntas sobre trading
 - Recibir asesoramiento personalizado
 - Ver historial de conversaciones

Perfil Personal

- Aquí puedes:
 - Ver tu información personal
 - Editar tu perfil
 - Cambiar contraseña

Arquitectura Técnica

Stack Tecnológico

Frontend:

- ☐ **Next.js** 14 (React)
- ☐ TypeScript
- ☐ Tailwind CSS
- ☐ NextAuth.js (Autenticación)

Backend:

- ☐ **Next.js** API Routes
- ☐ Node.js
- ☐ Prisma ORM

Base de Datos:

- ☐ PostgreSQL
- ☐ Prisma Client
- ☐ URL: `postgresql://postgres:Cjjunior1@localhost:5432/postgres`

Seguridad:

- ☐ NextAuth.js (JWT tokens)
- ☐ bcryptjs (password hashing)
- ☐ Email verification tokens
- ☐ **Role**-based access control (RBAC)

Flujo de Autenticación Técnico

1. Configuración de NextAuth (lib/auth-options.ts)

```
export const authOptions: NextAuthOptions = {
  providers: [
    CredentialsProvider({
      // Validar email y contraseña contra BD
      async authorize(credentials) {
        const user = await prisma.user.findUnique({
          where: { email: credentials.email }
        });

        // Verificaciones:
        // 1. Usuario existe
        // 2. Email verificado
        // 3. Contraseña válida (bcrypt)

        return {
          id: user.id,
          email: user.email,
          name: `${user.firstName} ${user.lastName}`,
          role: user.role // ← Se incluye el rol
        };
      }
    })
  ],
  session: {
    strategy: "jwt" // Usar JWT tokens, no sesiones en servidor
  },
  callbacks: {
    // JWT callback: Agregar rol al token
    async jwt({ token, user }) {
      if (user) {
        token.role = user.role;
      }
      return token;
    },
    // Session callback: Agregar rol a la sesión
    async session({ session, token }) {
      if (session.user) {
        session.user.role = token.role;
      }
      return session;
    }
  },
  pages: {
    signIn: "/login" // Página de login personalizada
  }
};
```

2. Flujo de Envío de Credenciales

```

Cliente Browser
↓
Página de Login (/app/login/page.tsx)
↓
signIn("credentials", { email, password })
↓
NextAuth.js Provider
↓
POST /api/auth/callback/credentials
↓
authOptions.authorize()
↓
Prisma Query: findUnique({ email })
↓
PostgreSQL Database
↓
Validación: bcrypt.compare(password, hashedPassword)
↓
JWT Token Generado
↓
httpOnly Cookie (seguro)
↓
Redirigir según rol

```

3. Verificación de Rol en Rutas Admin

```

// En: /lib/admin.ts
export async function requireAdminSession() {
  const session = await getServerSession(authOptions);

  // Si no está autenticado → login
  if (!session) redirect("/login");

  // Si no es admin → dashboard regular
  if (session.user.role !== "admin") {
    redirect("/dashboard");
  }

  return session; // Admin autenticado ✓
}

```

Endpoints API Relevantes

Endpoint	Método	Descripción
/api/auth/signin	POST	Login (NextAuth)
/api/auth/signout	POST	Logout
/api/auth/session	GET	Obtener sesión actual
/api/auth/login	POST	Validación de login
/api/signup	POST	Registro de usuario
/api/verify-email	GET	Verificación de email



Estructura de la Base de Datos

Tabla: User

```
CREATE TABLE "User" (
  id VARCHAR(24) PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  firstName VARCHAR(255) NOT NULL,
  lastName VARCHAR(255) NOT NULL,
  whatsappNumber VARCHAR(20) NOT NULL,
  role VARCHAR(50) DEFAULT 'user',
  emailVerified TIMESTAMP,
  verificationToken VARCHAR(255) UNIQUE,
  tokenExpiry TIMESTAMP,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Relaciones

```
User
├─ ChatSession (1:N)
├─ UserCourseAccess (1:N)
└─ UserProgress (1:N)
```



Crear Nuevos Usuarios

Método 1: A Través del Panel Admin (Recomendado)

1. Login como admin
2. Ve a /dashboard/admin/users
3. Haz clic en "Crear Usuario"

4. Completa el formulario:
 - Email
 - Nombre y Apellido
 - Número de WhatsApp
 - Selecciona rol (admin o user)
5. Haz clic en “Crear”

Método 2: Registro Público

1. Ve a `/registro`
2. Completa el formulario:
 - Email
 - Nombre y Apellido
 - Número de WhatsApp
3. El usuario recibe un email de verificación
4. Verifica el email haciendo clic en el enlace
5. Recibe contraseña temporal por WhatsApp
6. Puede hacer login en `/login`

Método 3: Línea de Comandos (Node.js)

```
const { PrismaClient } = require('@prisma/client');
const bcrypt = require('bcryptjs');

const prisma = new PrismaClient();

async function createUser() {
  const hashedPassword = await bcrypt.hash('YourPassword123', 12);

  const user = await prisma.user.create({
    data: {
      email: 'nuevo@ejemplo.com',
      password: hashedPassword,
      firstName: 'Juan',
      lastName: 'Perez',
      whatsappNumber: '+5219876543210',
      role: 'user', // o 'admin'
      emailVerified: new Date() // Ya verificado
    }
  });

  console.log('Usuario creado:', user);
  await prisma.$disconnect();
}

createUser();
```



Consideraciones de Seguridad



Implementado

1. **Contraseñas Hasheadas:**
 - Algoritmo: bcryptjs con salt rounds = 12
 - No se almacenan contraseñas en texto plano

2. Email Verificado:

- Se requiere verificación de email antes de login
- Token de verificación con expiración de 24 horas

3. JWT Tokens:

- Almacenados en httpOnly cookies
- No accesibles desde JavaScript (XSS protection)
- Firmados con NEXTAUTH_SECRET

4. Role-Based Access Control (RBAC):

- Admin requiere `role = "admin"`
- Verificado en el servidor en cada request

5. Rutas Protegidas:

- `/dashboard/admin/*` requieren autenticación admin
- Redirección a login si no está autenticado

Mejoras Recomendadas

1. Implementar 2FA (Two-Factor Authentication)

- SMS o TOTP

2. Rate Limiting:

- Limitar intentos fallidos de login
- Prevenir fuerza bruta

3. Auditoría:

- Registrar todos los logins
- Registrar cambios de rol
- Registrar acciones admin

4. Refresh Tokens:

- Implementar refresh tokens con expiración corta
- Mejorar seguridad de JWT

5. HTTPS Obligatorio:

- Solo en producción
- HSTS headers

Soporte y Problemas Comunes

Problema: “Credenciales inválidas” al hacer login

Soluciones:

1. Verifica que el email sea correcto (sensible a mayúsculas)
2. Verifica que la contraseña sea exacta
3. Asegúrate de que el email está verificado
4. Prueba a crear un nuevo usuario

Problema: “Por favor verifica tu email antes de iniciar sesión”

Soluciones:

1. Ve a la página de verificación

- 2. Si no recibiste el email, solicita uno nuevo
- 3. Revisa la carpeta de spam

Problema: No puedo acceder al panel admin

Soluciones:

- 1. Verifica que tu usuario tenga `role = "admin"`
- 2. Prueba a hacer logout y login nuevamente
- 3. Limpia cookies del navegador y vuelve a intentar

Problema: La contraseña no funciona

Soluciones:

- 1. Usa la contraseña que se te asignó (no puedes crear la tuya en registro)
- 2. Espera el email/WhatsApp con la contraseña temporal
- 3. Solicita un reset de contraseña (si está implementado)

 Resumen Ejecutivo

Aspecto	Descripción
Tecnología de Auth	NextAuth.js + Credentials Provider
Base de Datos	PostgreSQL + Prisma ORM
Hashing de Contraseñas	bcryptjs (salt rounds: 12)
Tipo de Sesión	JWT en httpOnly cookies
Roles Disponibles	admin, user
Protección de Rutas	middleware <code>requireAdminSession()</code>
Verificación de Email	Requerida antes de login
Usuarios Actuales	2 (1 admin, 1 user)

Documento generado: 2 de mayo de 2026
Sistema: Trading Academy Next.js
Estado:  Completamente funcional y documentado